

reation/_index.md

Team Ownership

Features/capabilities Code Creation owns today:

- Code suggestions: completion + generation
- Code-related slash commands: /explain, /refactor, /tests, /fix
- Repository X-Ray
- Code suggestions context (e.g. open tabs) - partnered with Editor Extensions Group

Code Creation is also responsible for evaluations for each of these features which includes:

- Creating datasets in LangSmith and registering them in the Datasets repository
- Creating evaluators in Centralized Evaluation Framework
- Running evaluations

About Code Suggestions

One of the main features we work on in the Code Creation group is Code Suggestions. Here is some quick information to get you started with Code Suggestions.

- Code Suggestions Engineering Overview - Handbook technical overview of Code Suggestions
- Code Suggestions Guide - Handbook development guide for engineers
- Code Suggestions Quick Start - GitLab Documentation
- Difference between Code Completion and Code Generation - YouTube
- Code Suggestion Documentation - GitLab Documentation

A lot of the terms we use in this area sound similar and can be confusing at first. Here are the basic terms we use:

- Code Creation: The group name and a collection of features relating to providing AI generated code
- Code Suggestions: A feature within Code Creation that provides AI-generated code within an IDE
 - Code Completion: A short AI-generated suggestion intended to complete an existing line or block of code
 - Code Generation: A longer AI-generated suggestion intended to create entire functions, classes, code blocks, etc.
- Duo Chat: Another feature that interacts with GitLab Duo Chat to write new code, refactor existing code, or scan code for vulnerabilities

If it helps, here are these terms in a diagram:

mermaid

stateDiagram

direction LR

state "Code Creation" as creation

state "Code Suggestions" as suggestions

state "Code Completion" as completion
state "Code Generation" as generation
state "Duo Chat Features" as duo

creation --> suggestions
creation --> duo
suggestions --> completion
suggestions --> generation

Team Handles

Use this information to connect with the Code Creation group:

Category	Handle
GitLab Team Handle	@gitlab-org/code-creation/engineers
Slack Channel	gcodecreation
Slack Handle (Engineers)	@code-creation-engs

Team Members

The following people are permanent members of the Code Creation Team:

{{< team-by-manager-slug "mnohr" >}}

You can reach the whole team on GitLab issues/MRs by using the @code-creation-team handle.

Stable Counterparts

The following members of other functional teams are our stable counterparts:

Category	Counterpart
Product Manager	{{< member-by-name "Jordan Janes" >}}
Technical Writing	{{< member-by-name "Steve Cawood" >}}
UX	TBD
Support	TBD
AppSec	TBD

Other Related Pages

- Product Categories: Code Creation
- Direction: Code Suggestion Direction

Code Suggestion Dashboards

- 1. Code Suggestions Metrics (README) - usage, acceptance rate, latency, error rates, etc (Tableau)
- 1. General Metric Reporting - can find code suggestions rate limiting, X-Ray usage, etc (Tableau)
- 1. Log Visualization Dashboard,time:(from:now-6h,to:now))) - another view of latency, response codes, number of requests, etc (Kibana)
- 1. Code suggestions latency: Breakdown of server-side latency for code suggestions (Kibana)
- 1. X-Ray Dependency Parsing Errors Dashboard: Breakdown of parsing errors triggered by X-Ray dependency scanning (Kibana)
- 1. Metrics Dashboard (Grafana)
- 1. Error Budget (Grafana)

SECTION: engineering/ai/code-creation/engineering_overview.md

Introduction

Welcome to the technical overview of GitLab's Code Suggestions, a feature designed to enhance the coding experience by integrating advanced AI technologies directly within your development environment. This page serves as your guide to understanding the architecture and interactions behind our innovative Code Suggestions feature, which significantly streamlines coding processes through intelligent completions and generative coding capabilities.

At its core, Code Suggestions operates through a sophisticated workflow involving multiple components such as IDE extensions, the Language Server, GitLab Workhorse, and our AI Gateway, all culminating in providing you with real-time, context-aware coding suggestions. From simple code completions that speed up your typing tasks to complex code generations that craft entire code blocks, our system is designed to support a wide array of coding activities and enhance productivity.

Below, we detail each component's role in this ecosystem, describe the flow of data through our system, and explain how different types of coding interactions are handled to provide both quick suggestions and detailed code generation.

Code Suggestions Technical Overview

Code Suggestions generally follow the sequence described in the following diagram.

```
mermaid
sequenceDiagram
    actor USR as User
    participant IDE
    participant EXT as IDE Extension
    participant LS as Language Server
    participant GLW as GitLab Workhorse
    participant GLR as GitLab Rails
    participant AIGW as AI Gateway
```

participant LLM as Large Language Model

```
USR->>IDE: types: "def add(a, b)"
IDE->>EXT: notify about document change def add(a, b)
EXT->>LS: register document change def add(a, b)
LS->>GLW: sends code suggestion requests
GLW->>GLR: proxies code suggestion request
GLR->>GLW: enriches code suggestion request
GLW->>AIGW: enriched code suggestion request
AIGW->>LLM: enriched code suggestion request
LLM->>AIGW: "a + b"
AIGW->>GLW: suggestion: "a + b"
GLW->>LS: suggestion: "a + b"
LS->>EXT: triggers IDE code suggestion UI: "a + b"
EXT->>IDE: triggers IDE code suggestion UI: "a + b"
```

Components pictured on diagram are as follow:

1. IDE extensions: GitLab offers number of IDE extensions (aka plugins) that among other features also provide integration with Language Server
 1. VSCode Extension: <https://gitlab.com/gitlab-org/gitlab-vscode-extension/>
 1. JetBrains Extension: <https://gitlab.com/gitlab-org/editor-extensions/gitlab-jetbrains-plugin>
 1. NeoVim Extension: <https://gitlab.com/gitlab-org/editor-extensions/gitlab.vim>
1. Language Server: it is a unified way of delivering features that can be shared across different IDEs reducing duplication. Language Server is a component that uses the LSP protocol for communication with IDE extensions.
1. GitLab Workhorse - GitLab Workhorse is a smart reverse proxy for GitLab intended to handle resource-intensive and long-running requests.
1. GitLab Rails - main GitLab component providing majority of features.
1. AI Gateway - a standalone-service that will give access to AI features to all users of GitLab, no matter which instance they are using: self-managed, dedicated or GitLab.com. For more conceptual information refer to architecture blueprint
1. Large Language Model - a AI model that provides code generation capabilities

Code Suggestions includes two types of interactions:

- Code Completion: A short AI-generated suggestion intended to complete an existing line or block of code
- Code Generation: A longer AI-generated suggestion intended to create entire functions, classes, code blocks, etc.

Each code suggestion request is categorised into a single category. Request categorization is performed by the Language Server before request is sent to GitLab Workhorse. If categorization is not done by Language Server then this categorization is performed by GitLab Rails.

Code Completion

Code completion interaction is one of two code creation requests that can be triggered by IDE.

Its goal is to provide very fast responses (< 1 second) at the cost of smaller suggestion size, and less context awareness of surrounding source code or repository files.

By default, the code completion request is sent directly to the AI Gateway with direct connection details fetched from the GitLab Rails monolith, as shown in the sequence described below.

Alternatively, the request can be routed from the GitLab Rails monolith, as shown in the diagram in the Code Suggestions technical overview.

A request prepared by the Language Server is proxied in mostly unmodified form without any additional context being attached. GitLab Rails' role in this feature is limited to mostly an authorisation entity assuring that the given user is allowed to use the Code Suggestions feature.

Code Completion Direct Connection Diagram

mermaid
sequenceDiagram

```
actor USR as User
participant IDE
participant EXT as IDE Extension
participant LS as Language Server
participant GLR as GitLab Rails
participant AIGW as AI Gateway

USR->>IDE: types: "def add(a, b)"
IDE->>EXT: notify about document change def add(a, b)
EXT->>LS: register document change def add(a, b)
LS->>LS: triggers code suggestion request
alt there is unexpired direct connection details in cache?
LS->>LS: fetches direct connection details from cache
else
LS->>GLR: requests for direct connection details
GLR->>LS: returns direct connection details (AIGW url and token, expiry, model details)
LS->>LS: caches direct connection details, with 1 hour expiry
end
LS->>AIGW: sends code suggestion requests using direct connection details
AIGW->>LS: suggestion: "a + b"
LS->>EXT: triggers IDE code suggestion UI: "a + b"
EXT->>IDE: triggers IDE code suggestion UI: "a + b"
```

The components pictured on the diagram are described in the technical overview section.

Code Generation

Code Generation interaction is another type of code creation request that can be triggered by IDE. Its goal is to provide long and extensive responses generating complete blocks of code like functions or classes. It has a much longer response time than code

completions (up to 30 seconds). This type of code creation request takes extended context into account when resolving the user task. This context comes from current files in IDE as well as Repository X-Ray report.

```
mermaid
sequenceDiagram
    actor USR as User
    participant IDE
    participant PG as GitLab PostgreSQL DB
    participant GLR as GitLab Rails
    participant AIGW as AI Gateway
```

```
USR->>+IDE: types: "35; generate a function that transposes a matrix"
IDE->>+GLR: trigger code generation for line "35; generate function
GLR->>PG: fetch X-Ray report for project and language
PG->>GLR: xrayreports record
GLR->>GLR: include first 300 entities from xray report into code generation prompt
GLR->>-AIGW: trigger code generation "35; generate function
```

In above diagram some components (inc: GitLab Workhorse or Language Server) are omitted for brevity reasons. However high level flow of requests shown in technical overview section remains unchanged.

Repository X-Ray

Repository X-Ray is a feature that generates additional context data for code generation requests. This data is used to ground the AI model into the context of existing source code and align it with its private API as well as coding patterns.

Repository X-Ray report is generated as shown on following diagram:

```
mermaid
sequenceDiagram
    actor USR as User
    participant GIT as Gitaly
    participant GLR as GitLab Rails
    participant PG as GitLab PostgreSQL DB

    USR->>GLR: commits a change to the project's default branch
    GLR->>+GLR: triggers Repository X-Ray background job
    GLR->>GIT: fetches relevant files on default branch
    GIT->>GLR: file blobs
    GLR->>GLR: processes file blobs
    GLR->>-PG: upserts records to xrayreports
```

Components pictured on diagram are as follows:

1. Gitaly - an application that provides high-level RPC access to Git repositories.

1. GitLab PostgreSQL DB - relational database engine storing GitLab operational data.

Existing Repository X-Ray reports are included into code generation requests as shown in diagram at code generation paragraph.

Code Tasks

A user can also use one of the predefined chat commands to suggest changes in the selected code.

We currently support refactoring,
explaining code,
and writing tests.

These commands can be used in Duo Chat and also its response is displayed in Duo Chat window.

mermaid

sequenceDiagram

participant USR as User

participant GLR as GitLab Rails

participant AIGW as AI Gateway

participant LLM as Large Language Model

Note over USR,LLM: GraphQL API

USR->>GLR: opens Duo Chat and sends /refactor messages with selected code

GLR->>USR: accepts the request and enques it

Note over USR,LLM: Asynchronous job

GLR->>AIGW: builds prompt and sends AI request

AIGW->>LLM: sends AI request

LLM->>AIGW: receives response

AIGW->>GLR: receives response

GLR->>USR: sends websocket message with response which is displayed in chat

API Reference

For a complete overview of the Code Suggestion API Endpoints that are used on both Rails Monolith and the AI Gateway Project, please refer:

- Rails app Code Suggestions API Reference
- AI Gateway API Reference

SECTION: engineering/ai/code-creation/code-suggestions-guide/_index.md

Introduction

This document contains Code Suggestions development guidelines for engineers.

For an overview of Code Suggestions, please refer to Code Creation Group engineering guide

Supporting new AI models

As the Code Suggestions offering continues to mature and we discover more about our users' needs as well as available AI models, we will need to add or switch to new AI models that Code Suggestions will use.

Integrating a new AI model into our systems generally consists of three steps:

1. Evaluation - to make sure that the AI model satisfies the requirements for our use cases.
2. Implementation - once we have evaluated the AI model as satisfactory, we need to update our code base accordingly.
3. Rollout - after we have added support for the new model, we need to follow a standard rollout process to slowly introduce it to our users.

SECTION: engineering/ai/code-creation/code-suggestions-guide/implementation_guidelines.md

These are guidelines for supporting a model for Code Suggestions in the AI Gateway (AIGW) and/or GitLab Rails.

Overview

Code Suggestion requests can be routed direct to the AI Gateway or indirect through GitLab Rails.

- For direct-to-AIGW requests, the IDE gets the model details from GitLab Rails through the Direct Connections API endpoint.
The IDE then sends a request to AIGW with the model details fetched from the GitLab Rails.
- For indirect-through-GitLab-Rails requests, the IDE sends a request to GitLab Rails' Code Completions API endpoint.
GitLab Rails then sends a request to the AIGW.

For a more in-depth overview of Code Completions vs Code Generations, and direct-to-AIGW vs indirect-through-GitLab-Rails requests, please refer to the Code Suggestions Technical Overview and the Code Completion guides.

AI Gateway

AI Gateway repository.

This is where the request to the AI model is implemented.
The AIGW serves as the hub between AI models and the rest of GitLab's systems.

You can introduce a new model in the AIGW without making it generally available to all GitLab users.

For example, you can introduce a new model for latency testing purposes. A new model only becomes available to GitLab users when it is enabled through GitLab Rails.

AIGW API endpoints

Code Completions and Code Generations requests to the AI Gateway are sent through the different Code Suggestions API endpoints. See the [Code Suggestions API documentation](#) for further information about AIGW's Code Suggestions API.

Considerations when adding a new model

- Make sure that you have specified stop tokens in the request to the model. (See [example MR](#).)
- Depending on the model and use case, you may need to include additional context to the request. This can involve adding the content of open files, such as project libraries.
- Most models will require post-processing, such as trimming whitespaces or removing comments. You can use existing post-processors ([example MR](#)) or add a new one ([example MR](#)) as needed.

GitLab Rails

GitLab Rails repository.

GitLab Rails does not send requests directly to an AI Model. It sends requests to the AIGW, which in turn sends requests to AI models.

GitLab Rails is the source of truth for which model to use for Code Completions or Code Generations. It toggles the current model through feature flags.

For direct-to-AIGW requests, GitLab Rails specifies the model details through the Direct Access endpoint.

For indirect-through-GitLab-Rails requests, GitLab Rails includes the model details in the payload for its request to the AIGW.

GitLab Rails API endpoints

- Direct Access endpoint -
for direct-to-AIGW requests, this endpoint provides the information necessary to send a request to AIGW
- Code Suggestions endpoint -
this is the endpoint used for indirect-through-GitLab-Rails requests

Generic guidelines for supporting a model

These are guidelines for when you are ready to support a model for Code Suggestions, for example, after you have done evaluations and have considered the model to be acceptable.

Create an epic

Introducing a model usually spans several tasks, so it is best to create an epic or sub-epic for this work.

Create a rollout plan

You must create a rollout plan before deploying a new model.
Refer to the Rollout Guide for more details.

Introduce behind a feature flag

For both the direct-to-AIGW and indirect-through-GitLab-Rails requests, the decision on what model to use ultimately comes from GitLab Rails. When introducing a new model, you must create a beta type feature flag in GitLab Rails to toggle the enablement of the new model. This feature flag must have an accompanying rollout issue.

Allow customers to opt out

Some customers may not be able to switch to the new model within the rollout timeline. We can add an opt-out capability to give customers more time before switching to the new model. This can be done by introducing an ops Feature Flag. This should ideally be an opt-out flag (instead of opt-in), and should always be false by default.

The Feature Flag actor can be decided on a case-by-case basis, but in general:

- On GitLab SaaS, the flag will be checked against the top-level group that is providing the user with the GitLab Duo Add-on seat.
- On Self-Managed GitLab instances, the flag can be checked on the instance level.

SECTION: engineering/ai/code-creation/code-suggestions-guide/model_evaluation_guide.md

This document serves as a technical how-to guide for evaluating new Code Suggestions models.

Evaluation template

When starting a model evaluation process, you must create an issue using the Model Evaluation Template.

Evaluation criteria

Before supporting a model for Code Suggestions, we must evaluate that model against several criteria, including correctness and latency.
For a more detailed list of criteria to consider, please refer to the evaluation template.

Evaluating correctness

To evaluate model correctness, use ELI5.

Evaluating latency

To evaluate model latency, use either ELI5

or the ai-model-latency-tester.

When evaluating by latency, it is recommended to check requests coming from different regions. The common regions to test are: North America, Europe, and APAC.

We can evaluate latency in the following ways:

- Direct to provider
 - Sending requests directly to the AI model provider, for example Vertex AI or Anthropic.
- Routed through AIGW to provider
 - Sending requests to the AIGW, which in turns sends requests to the provider.
 - Before this can be done, you will need to implement the model in the AIGW.
You can implement a model in the AIGW without making it generally available to GitLab users.

Evaluation methods

Evaluating by ELI5

ELI5 (Eval like I'm 5) provides a structured way to evaluate AI models using LangSmith. The ELI5 repository includes evaluation scripts, while the sample datasets and the result from the evaluations are stored in the LangSmith platform.

Running and analyzing evaluations on ELI5

For guidance on:

- Running evaluations, see the ELI5 documentation.
- Evaluating correctness and latency, see the Analyzing results documentation.

Running evaluations on a GCP instance

Running ELI5 evaluations on a GCP instance is ideal for getting consistent latency values that are not affected by your internet connection or your current location. Currently, there is no automated way to run evaluations on a GCP instance, so you must do this manually.

Please reach out to the gcodecreation Slack channel for guidance.

Evaluating by AI Model Latency Tester

The AI Model/Provider Latency Tester automates the evaluation of latency of third-party AI service providers, using clients in different geographic regions to simulate the experience of geographically dispersed users. It aims to assist in making data-driven decisions regarding which models should power GitLab's AI features.

See the Latency evaluations issue for further guidance and updates.

Evaluating by Load Tester

The AI Model/Provider Load Tester is designed to simulate production-like traffic, ensuring that the model provider can handle real-world workloads. During the model evaluation process, we should create and run the load test for the model.

For instructions on adding and running test scripts, please follow these steps.

SECTION: engineering/ai/code-creation/code-suggestions-guide/model_rollout_guide.md

This document serves as a guide for rolling out Code Suggestions models.

Create a rollout plan

Create an issue using the Rollout Plan Template.

This must be done before or during the implementation of the new model.

Specific rollout plans may need to be created for different rollout phases.

Rollout method

Rollout of a new model will be done through a beta feature flag.

Pre-rollout checklist

- Make sure that you have reviewed the considerations when adding a new model.
- Make sure that the model provider or host can handle the volume of requests on AI Gateway.

Rollout phases

Rollout of a new model is usually done in 3 phases:

1. Rollout to Code Creation team members and other interested stakeholders.
2. Rollout to all GitLab team members.
3. [Optional] Opt out selected customers from the new model
4. Rollout to all users.

Roll out to Code Creation team members and other interested stakeholders

This is an optional but recommended step before rolling out to all GitLab team members. Code Creation team members and interested stakeholders will have more knowledge of the currently used models, and they can give valuable feedback around latency and correctness in comparison to current models.

Before rollout, make sure that an internal feedback issue has been created. Make sure to direct all feedback to the internal feedback issue.

Roll out to all GitLab team members

This is a required step before rolling out to all users. With dogfooding from GitLab team members, the larger volume of requests can help us determine if:

- We need to increase the request quota for the model.
- Additional post-processing is needed for the completions coming from the model.
- We need to reach out to the model providers for bug fixes and other considerations.

Before rollout, make sure that the internal rollout and internal feedback issues have been created. Make sure to direct all feedback to the internal feedback issue.

[Optional] Opt out selected customers from the new model

Some customers may not be able to switch to the new model within the rollout timeline. We can opt them out by implementing an opt-out feature flag.

Ideally, we should enable this opt-out for the relevant customers before rolling out to all users.

Roll out to all users

This step is done when the model is ready for widespread use. Rollout to all users can be done by percentage of actors (recommended), or you can roll out to all users at once.

Since this is considered a feature release, ensure proper announcement of the release through appropriate channels before launch.

SECTION: engineering/ai/code-creation/code-suggestions-guide/testing_guide.md

This document serves as a technical how-to guide for testing Code Suggestions functionality.

End-to-End testing

Code suggestions is tested by using the API in `codesuggestionsspec.rb` and tested indirectly through the Web IDE in `codesuggestionsinwebidespec.rb`.

Code Suggestions Self-managed End-to-End Tests

In MRs, the end-to-end tests exercise the Code Suggestions API against self-managed instances by using an instance of the GitLab Linux (Omnibus) package integrated with the latest version of AI Gateway. The instance of AI Gateway is configured to return mock responses.

To view the results of these tests, open the `e2e:test-on-omnibus-ee` child pipeline and view the `ai-gateway` job. The `ai-gateway` job activates a cloud license and then assigns a Duo Pro seat to a test user, before the tests are run.

Note: The `e2e:test-on-omnibus-ee` pipeline either needs to be triggered manually or the MR needs

to have the pipeline:run-all-e2e label applied.

For further information, please refer to the GitLab QA documentation

Code Suggestions End-to-End Tests in live environments

Code Suggestion end-to-end tests run continuously against Staging and Production GitLab environments.

These tests run in scheduled pipelines and ensure the end-to-end code suggestion experience is functioning correctly.

Results can be viewed in the e2e-run-staging and e2e-run-production Slack channels. The pipelines can be found below, access can be requested in test-platform:

- Staging-canary pipelines
- Staging pipelines
- Canary pipelines
- Production pipelines

Running end to end tests locally

Using a GDK instance

Ensure that your GDK is connected to a working AI gateway instance, and that the root user has the ability to request code suggestions.

```
shell
cd qa
bundle install
QALOGLEVEL=DEBUG QAGITLABURL=https://gdk.test:3443 bundle exec rspec
qa/specs/features/ee/api/3create/codesuggestionsspec.rb --tag aigateway
```

The above command will log in as the default root user and create the appropriate personal access token to use in the test. This has to be done using the UI, so explicitly passing an admin access token will make the tests a lot faster:

```
shell
GITLABQAADMINACCESSTOKEN=<adminPAT> QALOGLEVEL=DEBUG QAGITLABURL=https://gdk.test:3443
exec rspec qa/specs/features/ee/api/3create/codesuggestionsspec.rb --tag aigateway
```

Using an Omnibus instance (orchestrated tests)

To run the AI gateway end to end tests (which includes code suggestion tests) as orchestrated tests, the following command will use the latest GitLab Linux (Omnibus) package and the latest AI Gateway image.

```
shell
CHROMEDISABLEDEVSHM=true DOCKERDEFAULTPLATFORM=linux/amd64 GITLABLICENSEMODE=
```

```
QAEEACTIVATIONCODE=<1Password> bundle exec gitlab-qa Test::Integration::AiGateway EE --no-teardown
```

Please see GitLab QA documentation for more details about AI Gateway scenarios.

Using staging.gitlab.com

The code suggestions end to end tests can be run against live environments. The following will execute the tests against staging:

```
shell
cd qa
bundle install
QALOGLEVEL=debug GITLABQAADMINACCESSTOKEN=<1Password> GITLABQAUSERAGENT=<1Pas
GITLABUSERNAME=gitlab-qa GITLABPASSWORD=<1Password> GITLABQAACCESSTOKEN=<1Passw
QAGITLABURL=https://staging.gitlab.com bundle exec rspec
qa/specs/features/ee/api/3create/codesuggestionsspec.rb --tag externalaiprovider
```

To find the secret values in 1Password, search for the variable name and the environment name, for example staging. Please see GitLab QA documentation for a complete list of supported environment variables.

SECTION: engineering/ai/code-creation/how-we-work/_index.md

Introduction

Welcome to the Code Creation group's working processes guide. This handbook page outlines how our team operates on a day-to-day basis, including our communication channels, issue tracking, development workflows, and milestone planning processes.

We're a globally distributed team working on AI-powered code creation features, and we've developed these practices to ensure clear communication, transparency, and effective collaboration across time zones. These processes help us maintain alignment with our goals while providing the flexibility needed for innovation and problem-solving.

This page serves as both an onboarding resource for new team members and a reference for existing team members to ensure consistency in our workflows. If you have questions about any of these processes, please reach out in our Slack channel (gcodecreation).

Commonly Monitored Issue Lists

- Planning Issues
- Issue board - by workflow
- Issue board - by project
- Issue board - by iteration
- Reliability

- Security
- InfraDev Issues

Engineering Onboarding

To help get started as a developer with the Code Creation team, we have created an onboarding issue template.

Time Off

Team members should add any planned time off in the `·Workday·` slack app, in accordance with the taking time off policy, including creating a PTO coverage issue.

Milestone Meetings

Our team operates on monthly milestones with one key meeting per milestone cycle. This meeting combines both the retrospective for the previous milestone and the kickoff for the upcoming milestone. To accommodate our globally distributed team, this meeting is held twice each milestone at different times - once in an APAC-friendly time zone and once in an EMEA/AMER-friendly time zone. This approach ensures all team members have the opportunity to participate live in the meeting that best fits their schedule.

The Combined Retrospective/Kickoff Meeting serves two purposes: reviewing the outcomes from our async retrospectives of the completed milestone, and aligning on objectives, discussing requirements, and assigning responsibilities for the upcoming milestone.

All of our meetings are recorded and uploaded to the Code Creation YouTube Playlist. Some meetings are marked as private, so internal team members will need to switch to use the Unfiltered YouTube account.

Milestone Planning

See the Code Creation Milestone Planning for more information on the milestone planning process.

Weekly Status Updates

We maintain a practice of weekly async status updates to ensure clear communication, track progress effectively, and maintain transparency across our team. This process aligns with our core values by fostering collaboration, driving results, and promoting efficiency through structured communication.

Timing and Frequency

- Team members post updates every Wednesday
- Updates are required for all assigned issues
- Multiple updates may be needed if working on multiple issues

Template

This is the template to use for the updates

markdown

Async Status Update yyyy-mm-dd

- Progress & Status: What progress have you made? What's the current state?
- Next Steps: What are your planned next actions?
- Blockers: Are you blocked or need assistance with this?
- How confident are you that this will make it to the current milestone?
 - ☐ Not confident
 - ☐ Slightly confident
 - ☐ Very confident

Remember to update the workflow label!

/cc @mnohr @jordanjanes

Be sure to tag the engineering manager, product manager, and any team members you are collaborating with.

Best practices

- Be specific and concise in updates
- Always include next steps, even if they're tentative
- Flag blockers early - don't wait until they become critical
- Use the template consistently for easier scanning
- Link to relevant issues or documentation when appropriate

SECTION: engineering/ai/code-creation/how-we-work/milestone_planning.md

At GitLab and on the Code Creation team, we believe in a structured yet flexible approach to milestone planning to ensure that our engineering and product teams are aligned, focused, and able to deliver high-quality work efficiently. This page outlines our process for planning and preparing for milestones.

Issue Creation

When creating a new issue, add as many details as possible during initial creation when knowledge is fresh. Assume low context for anyone reading or picking up the issue, and write down any information you think would be relevant.

Label new issues with `·workflow::refinement·` if they need further discussion or estimation.

Follow this template in the description:

markdown

Context

Summary or background of the feature, feature change, bug, or investigation. If applicable, include subsections such as "Further Details", "Scope", etc.

References and Resources

Include any links to documentations, other issues, or MRs that you think may be useful to the assignee. If applicable, indicate a domain expert or resource person/group.

Proposal

Proposed/suggested solution or any guidance related to it. For investigations or spikes, this may describe an expected outcome or expected output.

Definition of Done / Acceptance Criteria

List the specific conditions that must be met for this issue to be considered complete.

Issue Refinement

The goal of issue refinement is to have engineers review issues to ensure they are ready for development and provide lightweight estimates.

Use the template checklist in the next section as a guideline.

1. Initial Labeling:

- When a new issue is created or identified as a potential priority, it should be labeled with "workflow::refinement"
- This label indicates that the issue needs refinement/triaging before it's ready for development

2. Refinement Timing and Capacity:

- Refinement work is considered equal priority to development work
- Engineers should plan to spend approximately 20% of their time on refinement activities
- Focus on refining issues for the next 1-2 milestones to balance advance planning with changing requirements
- Avoid leaving all refinement work to the end of a milestone

3. Asynchronous Refinement Process:

- Team members, regardless of their timezone, can review issues with the "workflow::refinement" label at their convenience
- Use the issue comments for discussion and clarification. You may loop in other team members or domain experts from outside the team to provide further context and/or give feedback on a proposed solution.
- Individual team members are encouraged to schedule regular, asynchronous backlog refinement sessions on their calendars
- Use the Code Creation Refinement Board to make issues needing refinement easy to find
- Make sure to timebox your refinement efforts. Remember that the goal of refinement is not to provide a detailed solution, but to provide clarity on the problem and to provide a starting point on the solution. See Where to Draw the Line Between Refinement and Implementation.

4. Refinement Tasks:

- Team members should focus on the following when refining an issue:
 - Validate the problem is clear and aligned with roadmap priorities

- Ensure issue labels are accurate
- Clarify the issue title, description and requirements (including the Context, Proposal, and Acceptance Criteria sections)
- Break down the issue into smaller, manageable tasks if needed
- Identify and document any dependencies or blockers
- Estimate the effort required and assign a corresponding weight to the issue. Refer to issues estimation section for guidance.

5. Refinement Completion:

- Once a team member believes the issue has been sufficiently refined, they should:
 - Post the completed refinement checklist as a comment on the issue
 - Ensure that the issue has the appropriate weight
 - Add the "workflow::ready for development" label and remove the "workflow::refinement" label

Refinement Template

While refining an issue, copy the below template into a comment and work through each todo item. Once you've completed refinement, post the comment to document your refinement work:

markdown

Refinement Checklist

- [] Validated the problem is clear and aligns with team priorities, engaged in any discussions with the PM or other stakeholders as needed
- [] Updated title for clarity (if needed)
- [] Updated labels for proper categorization (if needed)
- [] Added or improved the "Context" section with necessary background information
- [] Added or improved the "Proposal" section with potential implementation approach or suggested ideas to further investigate
- [] Added or improved the "Acceptance Criteria" or "Definition of Done" section
- [] Identified dependencies or blockers (if any)
- [] Considered if the issue should be broken down into smaller tasks
 - [] Created child issues if needed (link them here)
- [] Assigned a weight using [issue estimation

/weight X

/label ~"workflow::ready for development"

Where to Draw the Line Between Refinement and Implementation

During refinement, focus on providing enough context and direction for someone else to implement the issue efficiently. While some investigation is often necessary during refinement, consider these guidelines:

- Refinement: Identifying the problem, proposing a general approach, providing helpful context, estimating effort, and setting success criteria.
- Implementation: Writing specific code, determining exact file locations, making detailed technical decisions, and executing the solution.

If your refinement work leads you to start implementing the solution, consider either:

1. Assigning yourself to the issue and transitioning to implementation
2. Documenting your findings without implementing, so someone else can benefit from your research

Remember that the goal of refinement is to make issues clear enough that any team member could pick them up and know what to do, without prescribing every implementation detail.

Milestone Planning

Once we have a sufficient pool of refined issues, we can proceed with milestone planning:

1. Capacity Planning:
 - The Engineering Manager calculates available capacity for the upcoming milestone using:
 - Planned time off
 - Historic velocity
 - Allocated refinement time
 - Other non-development activities (meetings, reviews, etc.)
2. Carryover Work:
 - Identify any issues that will not be completed in the current milestone
 - Adjust weights to reflect remaining work
 - Consider impact on next milestone's capacity
3. Refinement Prioritization:
 - The Engineering Manager and Product Manager identify and prioritize issues that need refinement for the upcoming 1-2 milestones
4. Issue Selection:
 - The Product Manager defines strategic priorities
 - Creates a prioritized list of issues for the upcoming milestone
 - Focus on issues that have already been refined when possible. These are issues with label "workflow::ready for development" and a weight.
 - If the PM identifies an issue that must go in the next milestone but is not yet labelled as "workflow::ready for development", they must flag that as early as possible and loop in a team member to refine the issue.
5. Issue Labels
 - The Product manager adds the correct code-creation-priority labels to indicate the importance of each issue
 - The Engineering Manager adds the "Deliverable" label to items we commit to focusing on in this milestone
 - Mark issues that exceed predicted capacity as "Stretch"

Role of Team Members

Engineers

- Actively participate in issue refinement activities
- Provide insights and recommendations on technical feasibility and implementation approach
- Share domain knowledge and expertise during refinement discussions
- Self-manage refinement work to meet the 20% time allocation target

Engineering Manager:

- Calculates team capacity and manages milestone planning
- Works with PM to prioritize upcoming issues
- Ensures refinement activities are properly distributed across the team

Product Manager:

- Defines strategic priorities for upcoming milestones
- Works with EM to prioritize upcoming issues
- Ensures issue requirements and acceptance criteria are clear
- Works with EM to identify future refinement needs for upcoming work

Issues Estimation

Here are the weights we use for issue estimation:

```
{{% include "includes/engineering/create/weighttable.md" %}}
```

A weight of 5 generally indicates the problem is not clear or a solution should be instead converted to an Epic with sub-issues. In these cases we may want to create a spike issue, which is an issue that is used to investigate and clarify the scope of the original issue. That spike issue can be used to help break down the original issue.

What To Work On

Work Prioritization

Issues are prioritized using a combination of two label types:

- The Deliverable label indicates work we commit to completing in the current milestone
- The code-creation-priority labels (high, medium, low) indicate the strategic importance of the work

The priority order for work is:

1. Issues labeled as both Deliverable and code-creation-priority::high
2. Other Deliverable issues (regardless of their priority level)
3. Non-deliverable issues based on their priority:
 - code-creation-priority::high
 - code-creation-priority::medium
 - code-creation-priority::low

What to Work on First: Deliverables

- Priority: Deliverables are considered the highest priority and should be completed before the end of the milestone, aligning with our iteration cycle and monthly release schedule.
- Self-Assignment: Engineers can pick any issue marked with the "Deliverable" label and assign it to themselves, making these issues the top priority for the milestone. Issues for the current milestone can be found on the Code Creation workflow board.

- Importance: It's crucial for engineers to make a concerted effort to complete these deliverables within the cycle, and to promptly communicate any obstacles to their engineering manager.
- Communication and Flexibility: Unexpected challenges can arise, potentially impacting the completion of a deliverable. Early communication of these challenges allows for possible solutions, such as scope reduction or reassignment. If a deliverable cannot be completed within the cycle, it is moved to the next cycle for completion. This scenario is an opportunity for both the engineer and the engineering manager to reflect and learn from the experience.

What to Work on Next

- Stretch Issues: After fulfilling deliverable responsibilities and other activities, engineers can use any remaining time to work on Stretch issues. These are lower priority and not expected to be completed within the current iteration cycle but are potential deliverables for the next cycle. Early progress on these issues is considered a bonus.
- Other Ideas: Engineers are encouraged to use spare time to contribute to any work they believe will significantly benefit the product or company. As the general guidelines state, "we recognize that inspiration is perishable, so if you're enthusiastic about something that generates great results in relatively little time feel free to work on that." While pursuing such initiatives, engineers should ensure proper issue assignment and consider sharing their work in relevant channels for visibility and collaboration.
- Responsibility over Rigidity: We expect people to be managers of one and prefer responsibility over rigidity, so there's no need to ask for permission if you decide to work on something that's not on the issue board, but please keep your other responsibilities in mind, and make sure that there is an issue, you are assigned to it, and you share it with the team.

Workflow and Label Management

- Label Updates: It's crucial for engineers to update the "workflow" labels accurately as issues progress from "ready for development" to "in dev" to "in review."
- Responsibility: Engineers are responsible for keeping the workflow labels up to date on all issues they are working on during the milestone.

This milestone planning process is designed to maximize our team's effectiveness by ensuring that everyone is aligned on priorities and responsibilities. It fosters a collaborative environment where every team member contributes to the planning process, leading to a more organized and productive development cycle.

Planning Metrics and Continuous Improvement

In order to measure the efficiency of this process we should track and regularly review the metrics

We track metrics such as:

- Average time an issue spends in refinement
- Number of issues refined per week
- Time spent on refinement activities vs. development

We also gather feedback from team members during retrospectives on the process.

SECTION: engineering/ai/data-science/_index.md

Vision

Build a diverse and global development team to support and drive results across the section, while maintaining our values and unique way of working.

Mission

Drive results through iterative development as we add AI, ModelOps and Anti-abuse features into the product.

Our teams are data-driven, support dogfooding, and collaboration within GitLab and the wider community.

Stages

AI-Powered
ModelOps

Career Development

{{% include "includes/engineering/foundations-data-science-shared/career-development.md" %}}

Team Days

{{% include "includes/engineering/foundations-data-science-shared/team-day.md" %}}

Metrics

{{% include "includes/engineering/foundations-data-science-shared/metrics.md" %}}

Development

{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}

 {{< tableau/filters "SECTIONLABEL"="datascience" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
 {{< tableau/filters "SECTIONLABEL"="datascience" >}}
{{< /tableau >}}

Say/Do Ratio

{{% include "includes/engineering/foundations-data-science-shared/say-do-ratio.md" %}}

Meetings

Although we have a bias for asynchronous communication, synchronous meetings are necessary and should adhere to our communication guidelines. Some regular meetings that take place are:

Meeting (Internal Only, limited access)	Frequency	DRI	Topics
----- ----- -----			
----- -----			

SaaS Availability	Weekly	Infrastructure	Incident
corrective actions, Feature Change Lock status, Past Due Infradev, Error Budget, or Security updates			
Engineering Stage Review Handbook	Monthly	Michelle Gill	
Productivity metrics, Technical Roadmap updates			
Foundations and Data Science EM Meeting	Biweekly	Michelle Gill	Standards
discussions, questions, help needed, FYIs, process improvements, collaborating on engineering stage direction, team event planning, leadership communication dissemination, wider initiative brainstorming, OKRs			
Core Development Staff Meeting	Biweekly	Tim Zallmann	Recurring
staff meeting for updates across engineering management within Tim's organization			
AI - Eng Leadership Sync	Weekly	Tim Zallmann	Engineering
coordination on shared AI objectives (architecture, inference, prompt migration) across groups and sections			

Stage Reviews

```
{{% include "includes/engineering/foundations-data-science-shared/stage-review.md" %}}
```

Skip Levels

```
{{% include "includes/engineering/foundations-data-science-shared/skip-levels.md" %}}
```

Links and resources

```
{{% include "includes/engineering/foundations-data-science-shared/shared-links.md" %}}
```

```
## SECTION: engineering/ai/data-science/ai-powered/_index.md
```

Vision

Build diverse and global development teams in the Data Science section to support GitLab's vision on the application of AI in the DevOps cycle,

while maintaining our values and unique way of working.

Mission

Drive results through iterative development as we add AI features with flexible ML Models capabilities into the product.

Our teams are data-driven, support dogfooding, and collaboration within GitLab and the wider community.

Duo

We are the driving force behind Duo, supporting features directly or through the empowerment of other teams. For a single source of truth of feature use-cases that we enable, see this list.

AI Powered Operating Principles

See here.

Stage Meetings

Although we have a bias for asynchronous communication, synchronous meetings are necessary and should adhere to our communication guidelines.

Some regular meetings that take place for all AI teams on the shared AI-Powered Stage Calendar (Calendar ID: cn5pdr2i2i5bjhs8aopahcjt84@group.calendar.google.com) are:

Frequency	Meeting	DRI	Possible topics
Monthly	AI Powered Monthly	Michelle Gill & Roger Woo	Roadmap review, leadership feedback, stage-wide direction & strategy
Weekly	AI Management Strategy and Alignment	Michelle Gill, Jacki Bauer, Roger Woo	AI Powered stage-wide / Duo strategy, execution, collaboration and decision making

UX

UX Triage

The UX team will review and triage issues to determine the appropriate level of UX support.

When creating a new issue, add the UX Triage label if there is a change that impacts the user experience, or if for any other reason you would like it to be reviewed by the UX team.

The UX team reviews this list each week. If UX involvement is recommended, we will add the following labels to indicate the priority:

- UX Priority::1: UX involvement is highly recommended due to design complexity and high impact to users and/or the business.
- UX Priority::2: UX involvement is recommended due to moderate complexity and impact to users and/or the business.
- UX Priority::3: UX involvement is recommended, time permitting. This is for issues where a designers input will improve the experience or the clarity of the solution, but the risk of a

negative impact to users and the business is low.

These labels do not block engineering from proceeding to work on the issue if they decide that delivery speed is more crucial to the business than UX quality.

In addition to UX Priority labels, the UX team will comment on the issue with an assessment of effort/time required so that engineering can make a decision about whether to proceed with or without UX input.

Stage Groups

SECTION: engineering/ai/data-science/ai-powered/ai-framework/_index.md

Vision

The AI Framework group is focused on how to support other product groups at GitLab with the AI Abstraction Layer, and GitLab AI feature development.

- Team OKRs

If you're interested in the team's Objectives and Key Results (OKRs), you can find them on GitLab.

- Team Members

Engineering Manager & Engineers

{{< team-by-manager-slug "davidoregan" >}}

Product, Design & Quality

{{% engineering/stable-counterparts manager-role="Engineering Manager(.)AI Framework" role="AI Framework" %}}}

Team members who are supporting this team on a short-term basis are located here where "Work priorities" include "AI Framework."

- Team Responsibilities

Team responsibilities include

Facilitating the integration of AI capabilities throughout GitLab by leveraging the AI Abstraction Layer and AI Gateway.

Guaranteeing the presence of comprehensive global observability, monitoring, documentation, and enhancements in latency.

Providing essential support for our AI chat system framework.

Incorporating support for new AI providers when required.

Assisting with the production support and ensuring the readiness of the AI Gateway.

LLM inference support, including prompt engineering, response evaluation, fine tuning, evaluation, and more.

· How to reach us

Depending on the context here are the most appropriate ways to reach out to the IDE Group:

Slack Channel: [gaiframework](#)

Slack Groups: [@ai-framework](#) (entire team) and [@ai-framework-engs](#) (just engineers)

· Team Processes

· Regular team meetings

· Important: For every meeting, the AI Framework team's meeting document should be used, and filled with the meeting notes, as well as references to any other sync meeting agendas/notes/recordings which have recently occurred. This will make it easier for people to find any meeting notes.

Team Meetings

1. Weekly Refinement Assignment Meeting

When: Every Monday, alternating between 14:00 AM GMT+1 and 19:00 PM GMT+1

What: This meeting replaces the previous Work Assignment Meeting and focuses on refining issues. The Engineering Manager and Product Manager ensure all issues are properly refined.

Shared calendars

AI Framework PTO (Calendar ID:

[ceca9440729dba2cbd88b3117fa70839836fb5811cb072132b94c52f912a31bf5@group.calendar.google.com](#)

AI-Powered Stage Calendar (Calendar ID: [cn5pdr2i2i5bjhs8aopahcjt84@group.calendar.google.com](#))

AI Framework team members should sync your PTO events with AI Framework PTO calendar.

· Weekly EM Updates

Each week the team EM provides a Weekly Status update issue which aims to capture the most important items for the team to be aware of. These can be found [here](#).

· AI Framework Board Outline

Our workflow process involves a weekly board walk where we review issues with the Deliverable label. Here's how we organize our board:

How do people know what to work on?

An issue is ready to be taken by an assignee when it has all of the following:

Deliverable label

The current Milestone

Either workflow::ready for development or workflow::refinement label

Board Lists

1. Open ·: This list includes all identified issues. An engineering manager will be assigned if either the Milestone or the "workflow::ready for development" label is missing.
2. workflow::design ··: During this phase, issues undergo design refinement. After design considerations are integrated, the "ready for development" label is applied.
3. workflow::refinement ·: Issues in this stage are undergoing engineering refinement to ensure the proposed solution meets all requirements. Once refined, the "ready for development" label is applied.
4. workflow::ready for development ·: Issues that are prioritized and assigned to a specific milestone are moved to this list, and the "ready for development" label is applied.
5. workflow::in dev ···: When a developer begins work on an issue, they should move it to this list and apply the "in dev" label.
6. workflow::in review ·: After development is complete, the issue moves to this list, and the "in review" label is applied.
7. workflow::verification ·: Following a successful code and UX review, the issue should be moved to this list and the "verification" label should be applied.
8. workflow::complete ·: Once the issue is verified and confirmed to be working properly, it should be moved to this list, the "complete" label should be applied, and the issue should be closed.

· Processes

·· Weekly Process

1. Open ·: (DRI: PM) Capture and list new issues as they arrive. This stage serves as the initial holding area for issues awaiting classification and prioritization.
2. Workflow Design ··: (DRI: Designer) Move issues into this lane for design refinement, focusing on user experience and interface design.
3. Workflow Refinement ·: (DRI: Engineering Lead) Conduct technical refinement to ensure feasibility and adherence to standards.
4. Workflow Ready for Development ·: (DRI: PM/EM) Transition fully refined and spec'ed issues here, ready for developers to start coding.
5. Workflow in Development ·: (DRI: Assigned Developer) Implement and build the specified features and functionalities.
6. Workflow in Review ·: (DRI: QA/Reviewer) Perform thorough reviews and testing, including code reviews and usability testing.

· Issue Guidelines

These guidelines apply to all issues we use for planning and scheduling work within our group. Our Engineers can define specific implementation issues when needed, but the overall goal for our issues are as follows:

Provide a meaningful title that describes a deliverable result.

- Add the new GitLab Duo chat package as a Vue2 extension
- Chat: move away from using OpenAI embeddings
- Make Chat better

Provide a meaningful description that clearly explains the goal of the issue, and provide some technical details if necessary.

Should there be critical implementation steps or other useful ways to create small tasks as part of the issue, please use a checklist as part of the issue descriptions.

The issue should have the Deliverable label, milestone and workflow label assigned.

Design and frontend engineering use one issue. The same issues moves from workflow::design to workflow::refinement to workflow::ready for development. This ensures that there is a single source of truth for customer-facing issues. If a design issue is too large to be implemented, it may be promoted to an epic.

It's okay to create specific engineering-driven implementation issues for more complex features. These would be called Child Issues and they should always link back to their parent. If one issue would spawn many child issues, consider creating an Epic.

· Weighting and Estimation Process

Weight Guidelines

Issues are weighted using the Fibonacci sequence (0, 1, 2, 3, 5, 8, 13+):

Weight 0: Smallest issues (typos, minor formatting, simple code changes without tests)

Weight 1: Simple issues with minimal uncertainty (good for new contributors)

Weight 2: Straightforward issues requiring multiple code/test updates

Weight 3: Larger issues with some complexity but manageable scope

Weight 5: Should typically be broken down; acceptable for large manual updates with low risk

Weight 8/13+: Placeholder weights indicating need for breakdown; too large or uncertain for immediate implementation

Weight Update Process

Every issue assigned to the upcoming milestone needs to be weighed before applying the Deliverable label by Engineering Manager. Engineering Manager needs to check whether weight is assigned and, in case of the weight being equal or above 5, works on breaking issues down into smaller ones.

Engineering manager and Product Manager are responsible for asking to weight issues assigned for the upcoming milestone during weekly team meetings. They should ask engineers to read issue descriptions before the meeting so they are ready to weight them and ask questions if needed. They can split this process between more than one meeting.

· AI Feature Evaluations Guidelines - Evaluate like I am 5

See the Evaluate like I am 5 Project and read the docs [here](#).

· Communication

The AI Framework Team communicates based on the following guidelines:

1. Always prefer async communication over sync meetings.

1. Don't shy away from arranging a sync call when async is proving inefficient, however always

record it to share with team members.

1. By default communicate in the open.

1. All work-related communication in Slack happens in the gaiframework channel.

- Time Off

Team members should add any planned time off in the "Workday" slack app, so that the Engineering Manager can use the proper number of days off during capacity planning.

- Ad-hoc sync calls

We operate using async communication by default. There are times when a sync discussion can be beneficial and we encourage team members to schedule sync calls with the required team members as needed.

- Other Useful Links

- AIGW Region Deployments

- Staging
- Production

- Dashboards (internal only)

Requests per provider

Error budgets

AI Gateway SLIs

GCP quota limits

LLM Sidekiq completions

Duo Chat Question Categorization

Security Issues

Reliability Issues

Sentry via CompletionWorker

Sentry via Feature Category

Monthly Retros

Chat QA Evaluation

Chat REST API Error Ratio

- GitLab Unfiltered Playlist

The AI Framework Group collates all video recordings related to the group and its team members in a playlist in the GitLab Unfiltered YouTube channel.

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
```

```
  {{< tableau/filters "GROUPLABEL"="ai framework" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-
```

```

west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="utai frameworkilization" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-
west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="ai framework" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-
2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard"
>}}
  {{< tableau/filters "GROUPLABEL"="ai framework" >}}
{{< /tableau >}}

```

SECTION: engineering/ai/data-science/ai-powered/custom-models/_index.md

Vision

The Custom Models group focuses on additional, custom models that power GitLab Duo functionality in support of our customers unique data and use-cases.

Organisation

The AI-powered:Custom Models team focuses on GitLab's suite of Custom Model features and is responsible for all backend aspects of the product categories that fall under the Custom Models group of the AI Powered stage of the DevOps lifecycle. Our Product direction is found on the Category Direction - Custom Models Management page.

The features we work with are listed on the Features by Group Page.

Team Members

Engineering Manager & Engineers

Engineering Manager: @timzallmann

Acting Manager: @eduardobonet

Name	Handle	team.yml	Email prefix
Igor Drozdov	@igor.drozdov	igor.drozdov	idrozdov
Patrick Cyiza	@jpcyiza	jpcyiza.yml	jpcyiza
Mohamed Hamda	@mhamda	mhamda.yml	mhamda
Julie Huang	@juliehuang	juliehuang.yml	julhuang
Bruno Cardoso	@bcardoso-	bcardoso-.yml	bcardoso
Eduardo Bonet	@eduardobonet	eduardobonet.yml	ebonet
Manoj M J	@manojmj	manojmj.yml	mmj

| Cindy Halim | @cindy-halim | cindy-halim.yml | chalim |

Product, Design & Quality

Product Manager: @susie.bee

```
{{% engineering/stable-counterparts manager-role="Engineering Manager(.)Custom Models"
role="Custom Models" %}}
```

How to reach us

- Issue Tracker: ~group::custom models
- Slack Channel: gcustommodels
- Label Subscription: ~group::custom models

Organisational Labels

Issues owned by the Custom Models group should have these labels, as appropriate:

- ~"group::custom models"
- ~"devops::ai-powered"
- ~"section::data science"
- ~"Category:Model personalization"
- ~"Category:Self-Hosted models"

In addition, issues should contain the relevant ~type: and subtype labels.

Scoping Work using Epics and Tech Leads

Epics should be used as the primary definition of scope for any work item larger a single issue in size. This could be a new feature, a complex refactoring or bug. The issues in the epic constitute the entire scope of the work item, and when they are all closed the work is completed and the epic is closed. An Epic should enclose an iteration with that adds a clear improvement, but an Epic does not necessarily represent the whole feature, wich might require multiple Epics.

The technical ownership of individual work items as defined by an Epic, is delegated to a "Tech Lead", an engineer. The engineer is assigned to the Epic and ensures the scope is correct. The Tech Lea